Subrat Das — Premium Portfolio + Admin CMS

A world-class, recruiter-ready personal portfolio for **Subrat Das** (MCA student, KIIT University, Bhubaneswar) with a complete **Admin CMS** that lets you manage every piece of content from a dashboard — no code edits required.

> Built with React 19, Vite, TypeScript, Tailwind CSS v4, Framer Motion, React Router, React Hook Form, Zod, and a Supabase-compatible data layer.

✨ Features

Public Portfolio

- **Animated Hero** with a live canvas particle-network background, typing effect, availability status, floating stat cards, and social icons.
- **About, Skills** (animated, category-filtered cards), **Education** & **Experience** timelines.
- **Projects** with rich detail modals: screenshots, architecture diagrams, features, challenges, learnings, technologies, GitHub & live links, view counts.
- **Certifications** with verification links, **Achievements**, **Coding Profiles** (LeetCode, GitHub, etc.).
- **Live GitHub Integration** — auto-fetches pinned repos, followers, stars, language distribution, and recent commits from the GitHub REST API.
- **Blog**, **Testimonials**, **Gallery** (filterable, fullscreen lightbox).
- **Resume** download/view with download counter.
- **Contact form** that stores messages in the database.
- **AI Voice Assistant** — floating chat with speech recognition (input) + text-to-speech (output), answers questions about Subrat using live portfolio data, navigates sections, opens GitHub, downloads resume.
- Fully **responsive**, **accessible** (WCAG-minded: reduced-motion, semantic markup, keyboard-friendly), **SEO-optimized** meta tags, and **lazy-loaded** sections for performance.

Admin CMS (`/admin`)

Secure email/password authentication, protected routes, session management, logout.

Modules (full CRUD where applicable):

- Dashboard overview with live analytics & quick actions
- Profile, Hero, About, Skills, Education, Experience, Projects, Certificates, Achievements, Coding Profiles, Gallery, Testimonials, Blogs, Social Links, Resume
- **Dynamic Page Builder** — drag-to-reorder sections, show/hide, add custom sections (Publications, Patents, Talks…), **Save Draft**, **Publish**, **Preview**.
- **Messages** — read, reply (via mailto), mark read/unread, star, delete, **export CSV**.
- **Analytics** — page views, project views, resume downloads, contact requests, assistant interactions, traffic chart, top referrers, per-project view counts.
- **Audit Logs** — every create/update/delete/login is tracked.
- **Settings** — SEO title/description/keywords, theme colors, fonts, OG image, analytics toggle.

🔐 Admin Credentials (demo)

```
Email:    admin@subrat.dev
Password: admin123
```

Visit `/admin/login`.

🏗️ Architecture

```
src/
├── lib/
│   ├── types.ts          # Domain types (mirror the SQL schema)
│   ├── store.ts          # Reactive data layer (Supabase-shaped API over localStorage)
│   ├── seed.ts           # Initial content for Subrat Das
│   ├── github.ts         # GitHub REST API integration
│   ├── assistant.ts      # AI assistant logic + Web Speech API
│   └── fieldConfigs.ts   # Field schemas driving the generic CMS editors
├── hooks/
│   └── useStore.ts       # Reactive store hooks (useSyncExternalStore)
├── components/
│   ├── ui/               # Reusable primitives (Reveal, Modal, Skeleton, ParticleField…)
│   ├── public/           # Portfolio sections + SectionRenderer (dynamic page builder)
│   └── admin/            # AdminLayout, generic CollectionEditor, SingletonEditor, fields
└── pages/
    ├── PublicSite.tsx
    ├── AdminApp.tsx
    └── admin/            # Dashboard, PageBuilder, Messages, Analytics, AuditLogs, Login
```

Data layer → Supabase

`src/lib/store.ts` intentionally mirrors the Supabase client API (`from(table).select/insert/update/delete`). To switch to a real Supabase backend:

1. `npm install @supabase/supabase-js`
2. Replace the functions in `store.ts` with Supabase client calls.
3. Run `supabase/schema.sql` in your Supabase project (includes tables, foreign keys, indexes, **RLS policies**, and a storage bucket).
4. Swap `signIn/signOut/getSession` for `supabase.auth`.

The rest of the app needs **no changes**.

🗄️ Database

`supabase/schema.sql` contains the full PostgreSQL schema with:

- 20 tables (profile, hero, about, skills, education, experience, projects, certificates, achievements, gallery, testimonials, blogs, coding_profiles, social_links, contact_messages, resume, settings, sections, analytics, audit_logs)
- Foreign keys, indexes
- **Row Level Security** policies (public read, admin write)
- Storage bucket + policies for resume/images
- `updated_at` trigger

🚀 Getting Started

```
npm install
npm run dev      # start dev server
npm run build    # production build
npm run preview  # preview the build
```

🛠️ Tech Stack

Layer	Tech
Frontend	React 19, Vite, TypeScript, Tailwind CSS v4, Framer Motion, React Router, React Hook Form, Zod, Lucide React
Backend	Supabase (PostgreSQL, Auth, Storage, RLS) — schema provided; localStorage shim used for the live demo
Integrations	GitHub REST API, Web Speech API (voice assistant)

📝 Notes

- The hero uses a GPU-friendly canvas particle network (reduced-motion aware) rather than a heavy Three.js bundle for reliability and performance.
- All content is seeded with realistic data for Subrat Das and is fully editable from the CMS.
- GitHub data is cached for 30 minutes client-side to respect rate limits.

© Subrat Das. Built with ❤️ and React.